

Code of the Day

- What does the following function print?

```
printf ("%d", (-2 >> 31))
```

```
11111111 11111111 11111111 11111110 >> 31
```

```
printf ("%d", (+2 >> 31))
```

```
00000000 00000000 00000000 00000010 >> 31
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Kernel Scheduling

03/17/2025

Before we get started...

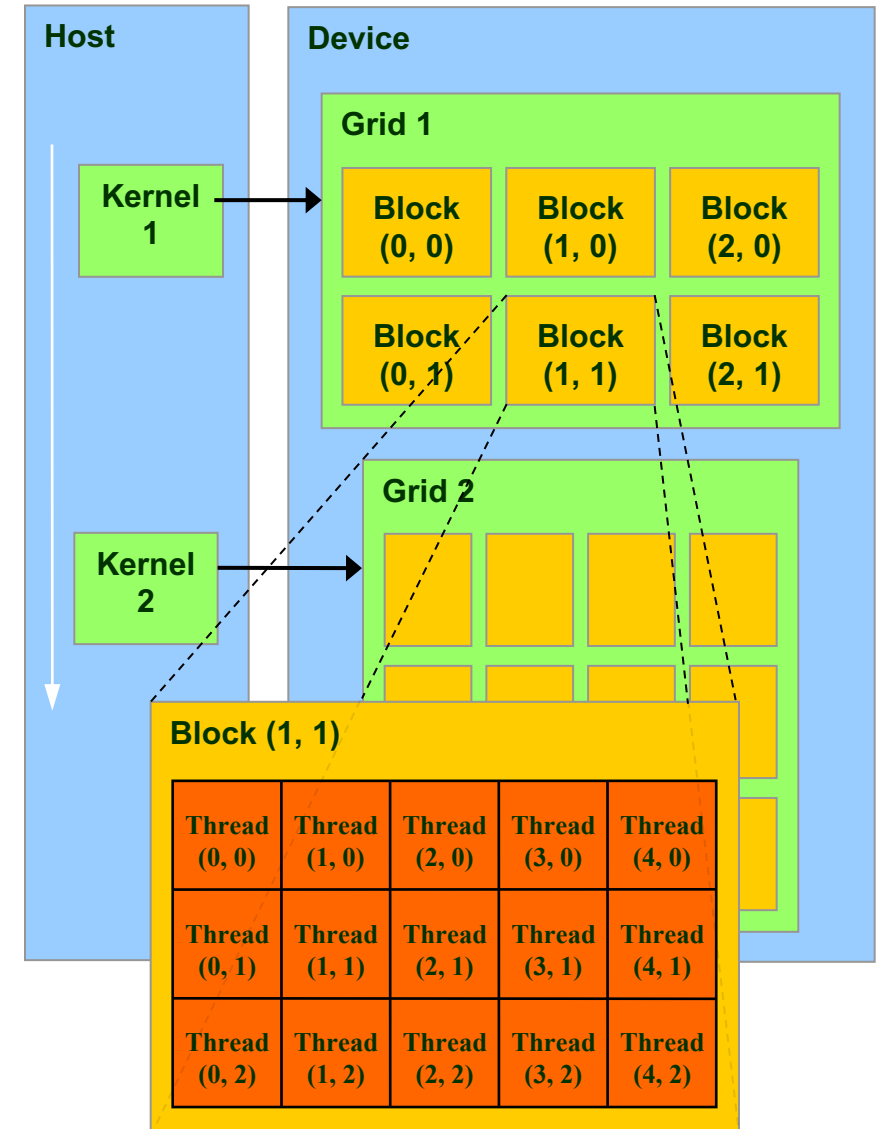
- Quick overview, things discussed last time
 - Kernel execution configuration
 - Went through a few examples, including a basic GPU-parallel matrix multiplication
 - Device-level scheduling for kernel
- Purpose of today's lecture: Cover the basics of GPU computing
 - Kernel scheduling
 - Thread divergence
- Miscellaneous
 - Assignment #4 – due on **03/17 at 23:50 PM**
 - Final project proposal – due on **03/24 at 23:50 PM**
 - Programming assignment #5 is released – due **3/31 at 23:50 PM**

Kernel Scheduling at a High-level Perspective

- There are two schedulers at work in GPU computing
 - A **device-level** scheduler: assigns one [more] block to an SM that signals room to run a block
 - This scheduler is also referred to as “GigaThread engine” by Nvidia
 - An **SM-level** scheduler: schedules the execution of the threads in a block onto the SM functional units
 - This is the more interesting scheduler that actually assigns threads to run your kernel code
 - Example functional units include FP16, FP32, INT32 processors, etc.

Device-Level Scheduler

- This is the first level of scheduling:
 - For running a large number of blocks given that we have a relatively small number of SMs
 - There are perhaps tens of thousands of blocks that will eventually get executed in 108 SMs (on an A100)
- Thread Blocks are distributed to the SMs
 - Potentially more than one block scheduled per SM
- As a Thread Block completes kernel execution on an SM, resources on that SM are freed
 - Device level scheduler will then dispatch next thread block in line
- There are limits for resident blocks and threads on an SM:
 - 32 blocks on the Pascal SM, Volta SM, and Ampere SM
 - No more than 2048 threads can be hosted on an SM

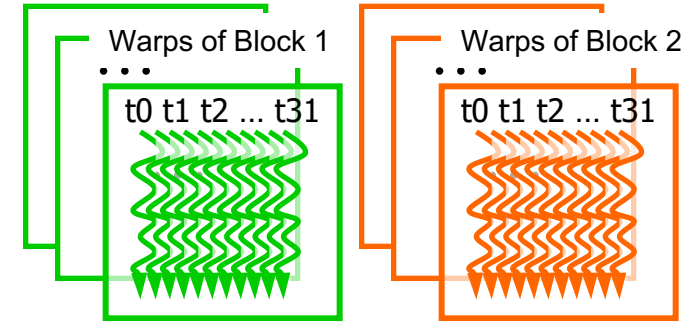


Device-Level Scheduler (cont'd)

- Once a block is picked up for execution by one SM, the block does not leave that SM before each thread of that block finishes executing the kernel
 - If the block has many threads idle due to boundary condition, you waste a lot of thread resources
- Once a block is finished & retired, only then can another block land for execution on that SM
 - The Device-Level Scheduler dispatches a block of threads to an SM that indicates that it has excess capacity
- Obviously, if your GPU has many SMs, many blocks will be running in parallel
 - The more expensive the GPU, the more SMs it has (just like a CPU having more cores)

SM-Level Schedulers

- Each block of threads is divided into 32-thread warps
 - “32”: selected by NVIDIA, programmer has no say
 - Warp: A group of 32 threads of consecutive IDs
- Warps are the basic scheduling unit on the SM
 - Everything eventually boils down to warp-level execution
- There is a limit on the number of resident warps per SM
 - 64: on Kepler, Maxwell, Pascal, Volta, Ampere (i.e., 2048 resident threads)
 - 48: on Fermi (i.e., 1536 resident threads)
 - 32: on Tesla (i.e., 1024 resident threads)
- Although in theory you can create many many threads ...
 - There are always hardware limits on the number of threads per block, number of blocks per SM, number of warps per SM, etc.



[short/important topic]: Thread Index vs. Thread ID

[critical in (i) understanding how SIMD is supported in CUDA, and (ii) understanding the concept of “warp”]

- Each block organizes its threads in a 3D structure defined by its three dimensions: D_x , D_y , and D_z that you specify.
- A block cannot have more than 1024 threads $\Rightarrow D_x \times D_y \times D_z \leq 1024$.
- Each thread in a block can be identified by a unique index (x, y, z) , and

$$0 \leq x < D_x \qquad 0 \leq y < D_y \qquad 0 \leq z < D_z$$

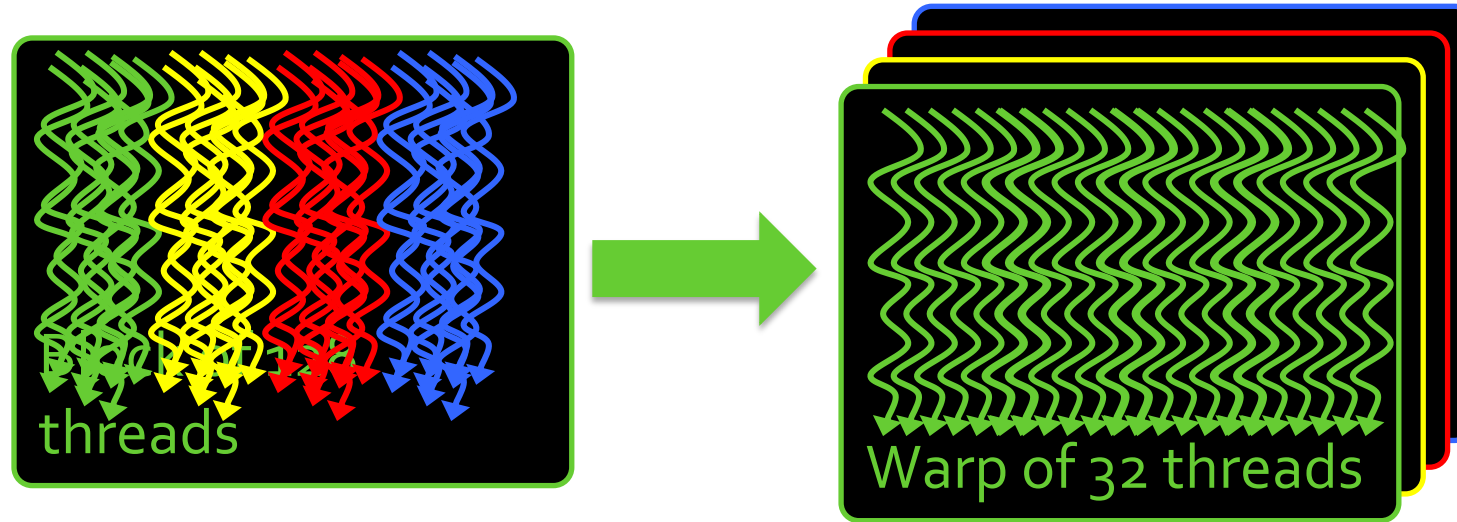
- A triplet (x, y, z) , called the thread index, is a high-level representation of a thread in the economy of a block. Under the hood, the same thread has a simplified and unique id, which is computed as $t_{id} = x + y * D_x + z * D_x * D_y$. You can regard this as a “projection” to a 1D representation. The concept of thread id is important in understanding how threads are grouped together in warps (more on “warps” later).
- In general, operating for vectors typically results in you choosing $D_y = D_z = 1$. Handling matrices typically goes well with $D_z = 1$. For handling PDEs in 3D you might want to have all three block dimensions nonzero.

Organizing Threads into Warps

- Thread IDs within a warp are **consecutive** and **increasing**
 - Related to the 1D projection from thread index to thread ID
 - In CUDA multidimensional blocks, the **x** thread index runs fastest, then **y**, then **z**
 - Threads with IDs (0...31) combine into warp 0, threads (32...63) into warp 1, etc.
- Partitioning of threads into warps is always the **same**
 - Warp size has always been 32 and is unlikely to change soon
- While you can rely on ordering among threads, **DO NOT** rely on any ordering among warps (there is no such thing)
 - Warp scheduling is not under user control in CUDA
 - Which warp runs first or later is completely decided by the SM-level scheduler

Blocks of Threads are Organized and Executed as Warps

- Each thread block split into one or more warps
- If thread block size is not multiple of warp size, unused threads go wasted
 - Example: block w/ 50 threads – takes 2 warps of 64 threads, of which the last 14 threads go wasted



- The hardware schedules each warp independently
- Different warps within a thread block execute independently

Different Compute Capabilities May Have Different Limits

GPU Features	NVIDIA Tesla P100	NVIDIA Tesla V100	NVIDIA A100
GPU Codename	GP100	GV100	GA100
GPU Architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
Compute Capability	6.0	7.0	8.0
Threads / Warp	32	32	32
Max Warps / SM	64	64	64
Max Threads / SM	2048	2048	2048
Max Thread Blocks / SM	32	32	32
32-bit Registers / SM	65536	65536	65536
Max Registers / Block	65536	65536	65536
Max Registers / Thread	255	255	255
Max Thread Block Size	1024	1024	1024
FP32 Cores / SM	64	64	64
Ratio of SM Registers to FP32 Cores	1024	1024	1024
Shared Memory Size / SM	64 KB	Configurable up to 96 KB	Configurable up to 164 KB

Quiz

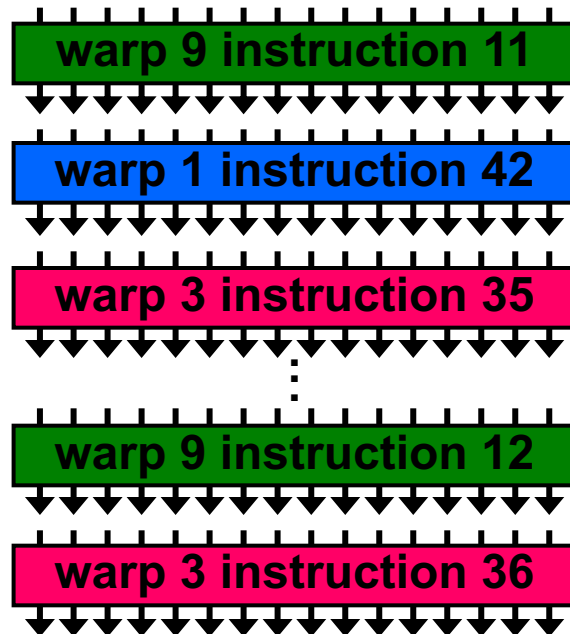
- If it turns out that 3 blocks are processed by an SM and each Block has 256 threads, how many warps are managed by the SM?
 - Each block of threads is divided into $256/32 = 8$ warps
 - There are $8 * 3 = 24$ warps
 - At any point in time, there are 24 warps that can be selected for execution
- NOTE: the SM can handle 2048 threads, yet you have it handle only 768. Occupancy is low

SM-Level Warp Scheduling is Very Fast



**SM multithreaded
Warp scheduler**

time

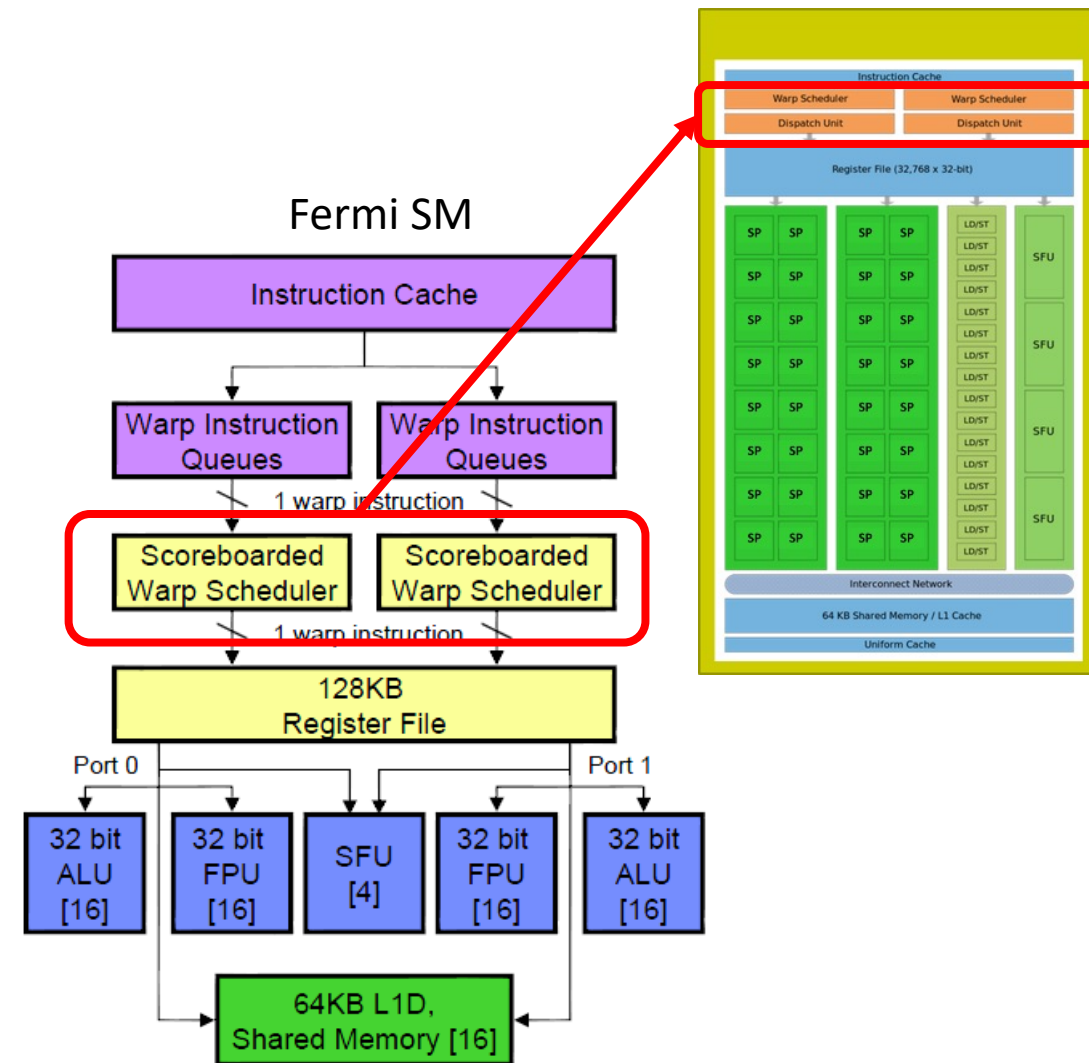


- SM hardware implements almost zero-overhead warp scheduling
- A warp whose next instruction has its operands ready for use is eligible for execution
- Eligible warps are selected for execution on a prioritized scheduling policy managed by hardware
- All threads in a warp execute the same instruction, in lockstep fashion – all threads complete that instruction together and then move on

Context switch overhead among warps is very low
(unlike CPU)

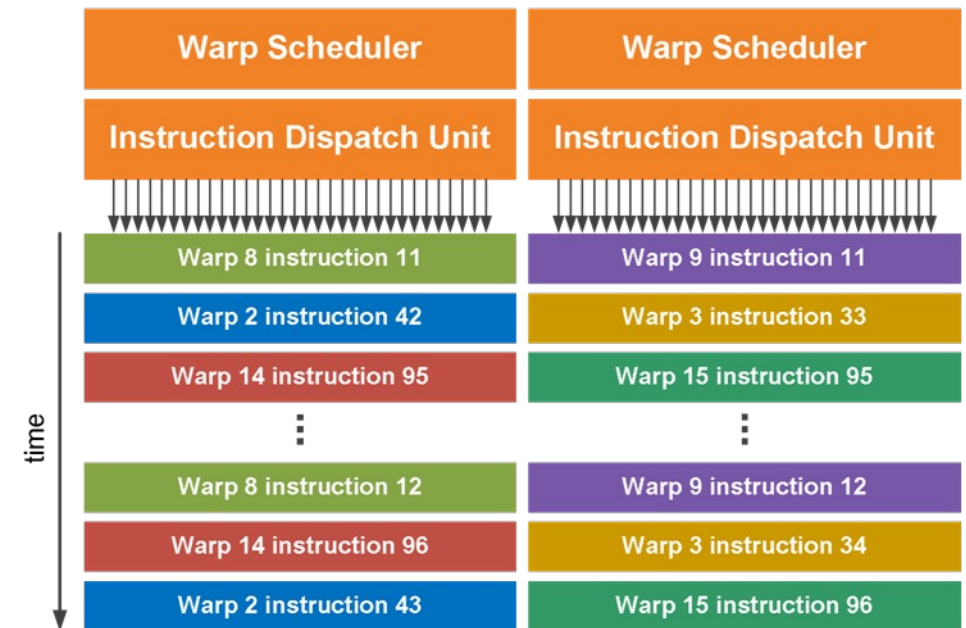
Fermi Specifics

- There are two warp schedulers per SM that can each issue one warp of threads
- One warp issued at each clock cycle by each scheduler (very fast!)
- No more than two warps can be dispatched for execution on the SM's functional units during any given cycle
- Scoreboarding used to figure out which warp is ready



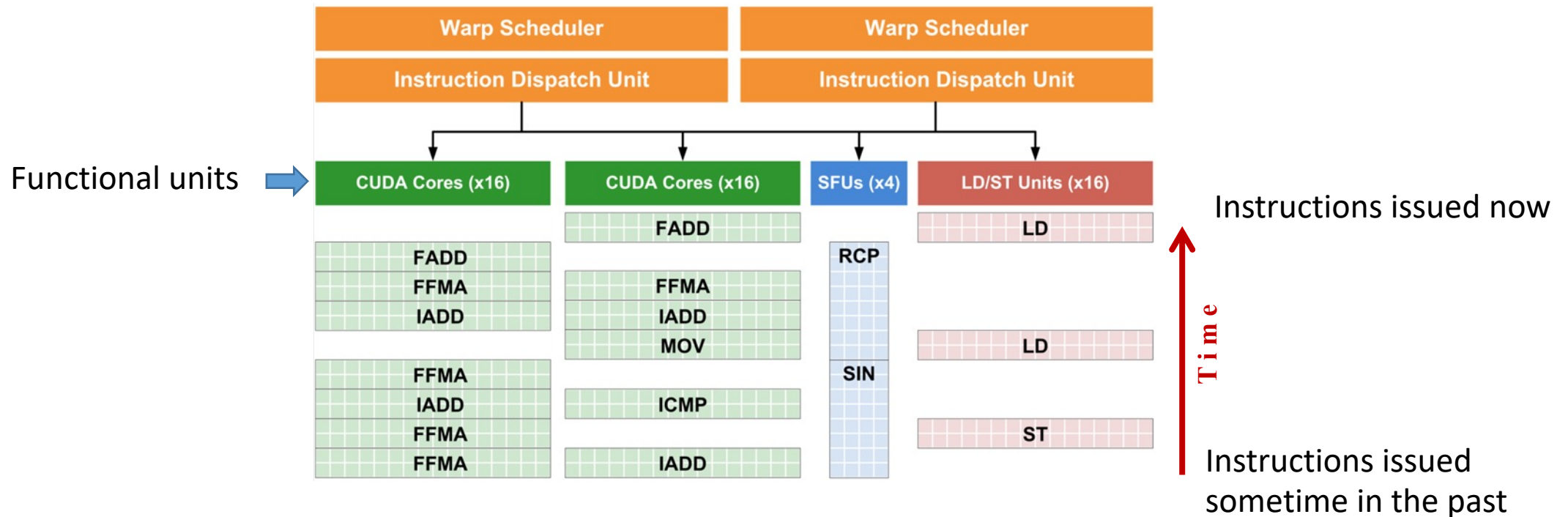
GPU Architecture is Pipelined

- One instruction can be retired with each clock cycle
 - Multiple instructions can be pipelined too
- However, ILP support is relatively limited
 - To the best of my knowledge:
 - There is no out-of-order execution smarts
 - There is no prefetching
 - There is no speculative execution
- The odd warp index are handled by a scheduler, the even ones are handled by the other scheduler
 - No exchanging warps at run time between schedulers



Fermi Specifics

- As shown in picture, at no time can we see more than 2 warps being dispatched for execution during a cycle
 - Note that at any given time we might still have more than two functional units working though (which is actually very good, device kept busy)



Good to know... When SM has multiple partitions (like Ampere)

- A warp always runs on one of the four sub-partitions
 - It does not change sub-partitions during execution
- Each sub-partition holds up to 16 warps
 - Per SM:
 - 4 partitions $\times$ 16 warps
 - 64 warps $\Rightarrow$ max 2048 threads



FP32 & INT32 instructions can be carried out at the same time

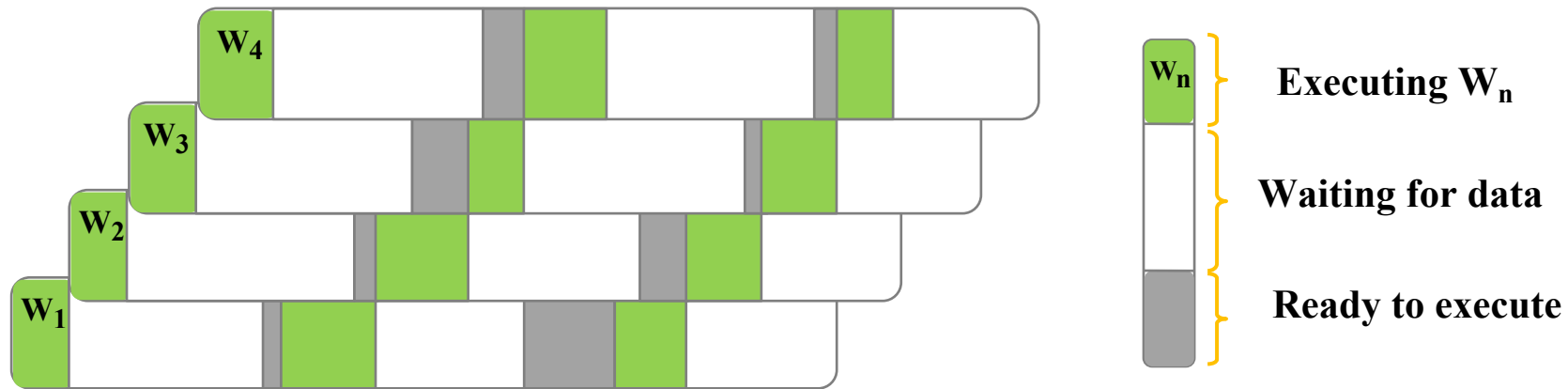
- The A100 SM includes separate FP32 and INT32 scalar processors
 - Allows simultaneous execution of FP32 and INT32 operations at high throughput
- How is this good?
 - Many applications have inner loops that perform pointer arithmetic (integer memory address calculations) combined with floating-point computations that will benefit from simultaneous execution of FP32 and INT32 instructions
 - Each iteration of a pipelined loop can update addresses (INT32 pointer arithmetic) and load data for the next iteration while simultaneously processing the current iteration in FP32
 - You don't have "structural hazards" (recall ILP discussion)



Why Many Warps in Flight is a Good Thing?

- An SM can switch between warps with almost no overhead (GPU has direct hardware to support warp switch)
- Warps with instruction whose inputs are ready are eligible to execute, and will be considered when scheduling
- When a warp is selected for execution, all [active] threads execute the same instruction in lockstep fashion

(pre-Volta only; this limitation lifted starting w/ Volta)



Warp scheduler handling four warps simultaneously

Possible states of a Warp

- First possible state: Active Warps (deployed on an SM)
 - Another possible sub-state: Eligible Warps (data dependency are met)
 - Another possible sub-state: Issued Warps (formerly Eligible, now executing)
 - Another possible sub-state: **Ineligible** Warp (waiting on data or functional units)
- Second possible state: Retired Warps (no more instructions left in the kernel for such a Warp)

What Cause an Active Warp to become Ineligible?

- An instruction fetch
 - Next instruction not available
- A memory dependency
 - Result of memory instruction, e.g., load, is not ready yet
- An execution dependency
 - Result of previous instruction is not ready yet
- A synchronization barrier (more on this later)
 - E.g., `__syncthreads()`

Execution Configuration to Maximally Utilize Warps

- Prefer thread block sizes that result in mostly full warps – assuming N elements

Bad: `kernel<<<N, 1>>> ( ... )`

Okay: `kernel<<<(N+31) / 32, 32>>>( ... )`

Better: `kernel<<<(N+127) / 128, 128>>>( ... )`

- Many threads per block provide SM with many warps to switch between
 - Keep in mind each SM can simultaneously schedule many warps in almost no cost
 - This is how the GPU hides memory access latency via throughput optimization
- Resource like ShMem amount & RegFile size may constrain number of threads per block
 - Resource per thread can access is limited by the hardware

Takeaway for GPU Scheduling: Device Level and SM Level

- When host invokes a kernel grid, the blocks of the grid are enumerated and distributed to SMs with available execution capacity
- Number of resident blocks on one SM has a certain limit depending on architectures
 - Up to 8 blocks on Fermi,
 - Up to 16 on Kepler & Turing
 - Up to 32 on Maxwell, Pascal, Volta , Ampere
- When a block of threads is executed on an SM, its threads are grouped in warps. The SM manages several warps at the same time
 - Up to 48 warps can be managed by an SM on Fermi (through 2 warp-level scheduler with fast switch)
 - Up to 32 warps can be managed on Turing
 - Up to 64 warps can be managed on Kepler, Maxwell, Pascal, Volta, Ampere
- When a thread block finishes, a new block is launched on the vacated SM

Thread Divergence

[pre-Volta]

Thread Divergence

- Consider the following code:

```
__global__ void odd_even(int n, int* x)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if( (i & 0x01) == 0 )
    {
        x[i] = x[i] + 1;
    }
    else
    {
        x[i] = x[i] + 2;
    }
}
```

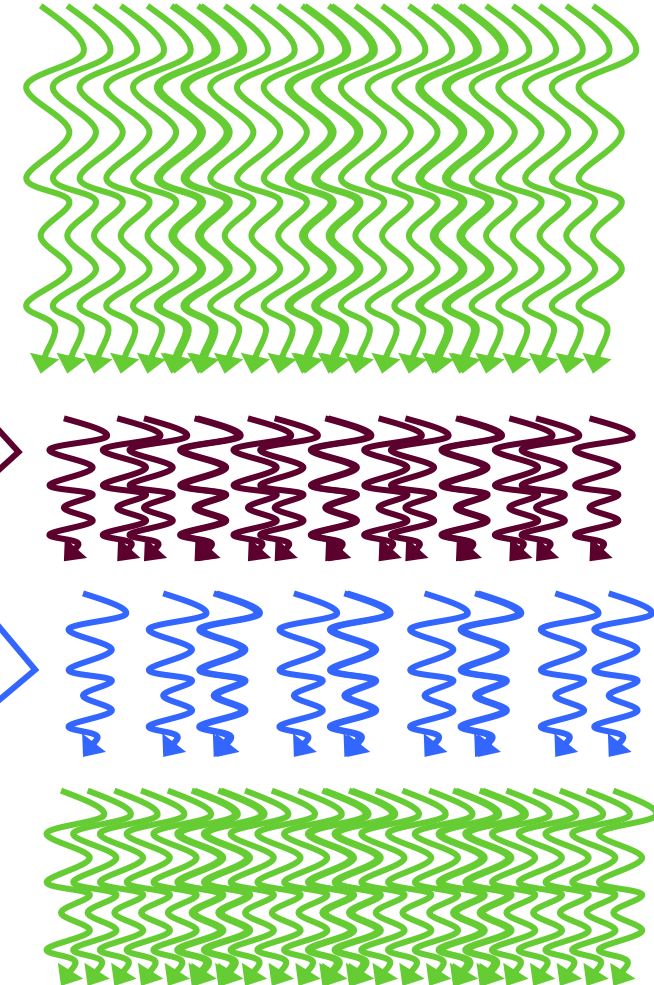
- Half the threads (even *i*) in the warp execute the **if** clause, the other half (odd *i*) the **else** clause

Thread Divergence in “if-then-else”

- Handling of an **if-then-else** construct in CUDA
 - First, a subset of threads of the warp execute one branch
 - Execution moves forward for these threads in lockstep fashion
 - Next, the rest of the threads in the warp execute the other branch
 - Execution moves forward for these threads in lockstep fashion
 - Notice that the above two executions do not run in parallel
 - As a result, for a two-branch if-else block, the parallelism is reduced by a factor of 2, even though the two branches do not have data dependency and can run in parallel
- **After** the if-then-else statement, all threads of the warp move on in **lockstep fashion**

Visualization of Thread Divergence

```
__global__ void odd_even(int n, int* x)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if( (i & 0x01) == 0 ) // even indices
    {
        x[i] = x[i] + 1;
    }
    else
    {
        x[i] = x[i] + 2;
    }
}
```



Takeaway of Thread Divergence

- Nested branches are handled similarly but the problem is further exacerbated
 - Deeper nesting results in more threads being temporarily disabled
- In general, one does not need to consider divergence when reasoning about the correctness of a program
 - Certain code constructs, such as those involving schemes in which threads within a warp spin-wait on a lock, can cause deadlock
- In general, one does need to consider divergence when reasoning about the performance of a program
 - Recall that a two-branch if-else block reduces the parallelism by a factor of 2
- NVIDIA calls execution model **SIMT** (Single Instruction Multiple Threads) to differentiate from actual SIMD where threads really are in lockstep

Quiz: Thread Divergence

- In an `if-then-else` construct for which thread divergence occurs, it's not clear whether the “`if`” part is executed first and then the “`else`” part is executed, or vice-versa
- Can you think of a scenario when this might cause problems?

```
// assume x[i] is initialized to all 1
__global__ void problem_of_thread_divergence(int n, int* x)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if( (i & 0x01) == 0 ) {
        x[i] = x[i] + 1;
    }
    else {
        x[i-1] = x[i] * 3;
    }
}
```

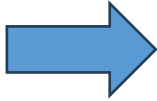
Performance of Divergent Code

- Performance decreases when the degree of divergence increases in warps
- Here's an extreme example that shows every thread is serialized ...

```
__global__ void dv(int* x)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    switch (i % 32)
    {
        case 0 : x[i] = a(x[i]);
                break;
        case 1 : x[i] = b(x[i]);
                break;
        ...
        case 31: x[i] = v(x[i]);
                break;
    }
}
```

Branchless Code Example

```
int abs(int x) {  
    if (x < 0) {  
        return -x;  
    }  
    else {  
        return x;  
    }  
}
```



```
int abs_branchless(int x) {  
    int mask = x >> (sizeof(int) * 8 - 1);  
    return (x + mask) ^ mask;  
}
```

Examples:

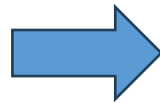
For $x = -5$: $\text{mask} = -1$, so $(-5 + -1) \wedge -1 = -6 \wedge -1 = 5$

For $x = 5$: $\text{mask} = 0$, so $(5 + 0) \wedge 0 = 5$.

1. $x \gg 31$ shifts the sign bit of x to the right by 31 bits, creating a mask. If x is negative, this results in all bits set to 1 (-1 in two's complement); if x is positive, this results in all bits set to 0.
2. Adding mask to x effectively subtracts 1 if x is negative, preparing it for the next step.
3. XORing $(x + \text{mask})$ with mask flips the bits if x was negative, giving you the absolute value of x without any branches.

Branchless Code Example 2

```
int max(int x, int y) {  
    if (x > y) {  
        return x;  
    }  
    else {  
        return y;  
    }  
}
```

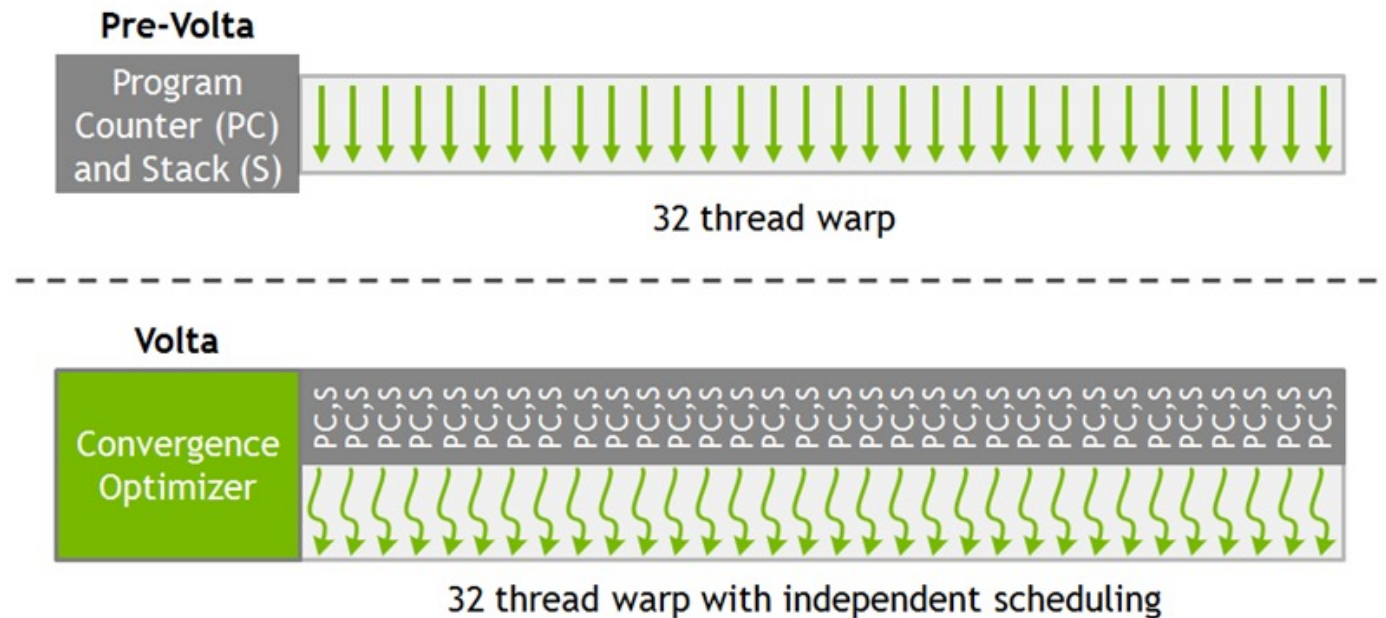


```
int max_branchless(int x, int y) {  
    int diff = x - y;  
    int mask = (diff >> (sizeof(int) * 8 - 1));  
    return x - (diff & mask);  
}
```

mask is: 0 if $x \geq y$ or -1 if $x < y$

Thread Divergence is Less Severe after VOLTA Architecture

- Volta GV100 was the first GPU to support independent thread scheduling, which enables *finer-grain* synchronization and cooperation between parallel threads in a program
- Threads can now diverge and re-converge at finer sub-warp granularity
 - Volta maintains execution state (program counter (PC) and call stack (S)) **per thread**



The NVIDIA GPU Memory Ecosystem

GPU Memory Types

- Register – dedicated HW on chip - single cycle NOTE: you don't have control over this
- Cache/Shared Memory on chip – a few cycles NOTE: physically, L1 & ShMem are same
- Local Memory – DRAM, (if not in cache - *slow*; if cached – relatively fast)
- Global Memory – DRAM, no cache - *slow*; if cached, is fast...
- Constant Memory – DRAM, cached, 10s...100s of cycles, depending on cache locality
- Texture Memory – DRAM, cached, 10s...100s of cycles, depending on cache locality
- Instruction Memory (invisible) – DRAM, cached

Storage Locations

Memory	Location	Cached	Access	Who
Register	On-chip	N/A	Read/write	One thread only
Shared	On-chip	N/A	Read/write	All threads in a block
Global	Off-chip	Yes	Read/write	All threads + host
Constant	Off-chip	Yes	Read	All threads + host
Texture	Off-chip	Yes	Read	All threads + host

Off-chip means on the GPU but not on the SM, which translates into higher latency.

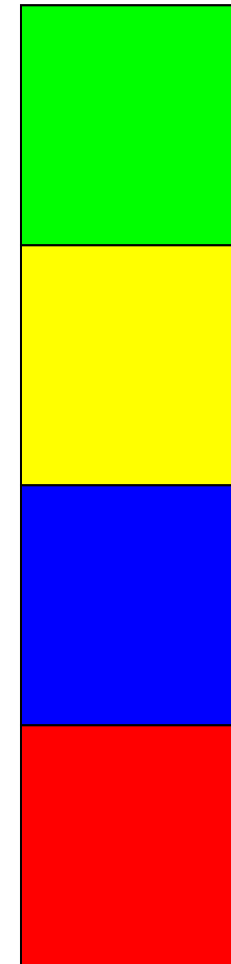
What “register” means in CUDA

- Any variable that you are using in a kernel *at some point* eats up a register
 - The variable that you use – it needs to “live” somewhere
 - Ex: `int a = 1;` the variable `a` will be stored in a register for completing the operation
- Every variable you use in a kernel will burn (at some point in the execution flow) a register
 - NOTE: even a variable stored in shared memory, for it to be operated on, it'll eat up a register* at some point in the execution
- As execution advances through a kernel, the number of registers occupied keeps changing
 - What's relevant: max number of registers required during the execution of the kernel
- *: except on Ampere, where you can control this behavior for further optimization

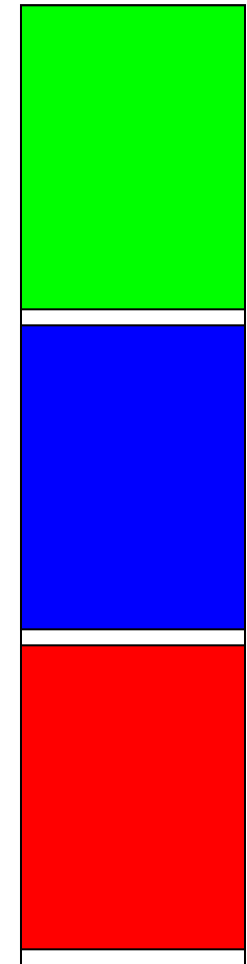
The SM's view of register file

- Number of **32 bit** registers in one SM is as follows:
 - 8K registers in each SM in G80
 - 16K on Tesla
 - 32K on Fermi
 - 64K on Kepler, Maxwell, Pascal, Volta, Ampere, and Hopper
- Registers are dynamically partitioned across all Blocks and assigned to the SM
- Once assigned to a Block, these registers are NOT accessible by threads in other Blocks
 - Yes, they can be accessible by threads in the same block using warp-level instruction primitives
- A thread in a Block can only access registers assigned to itself
 - **IMPORTANT:** A thread can have assigned by the compiler up to 255 registers – any thing beyond will cause the compilation to fail

4 blocks

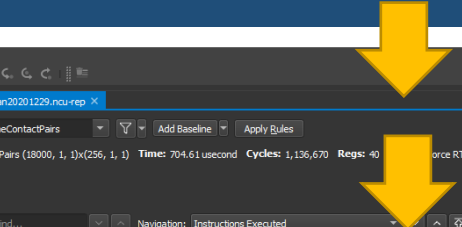


3 blocks

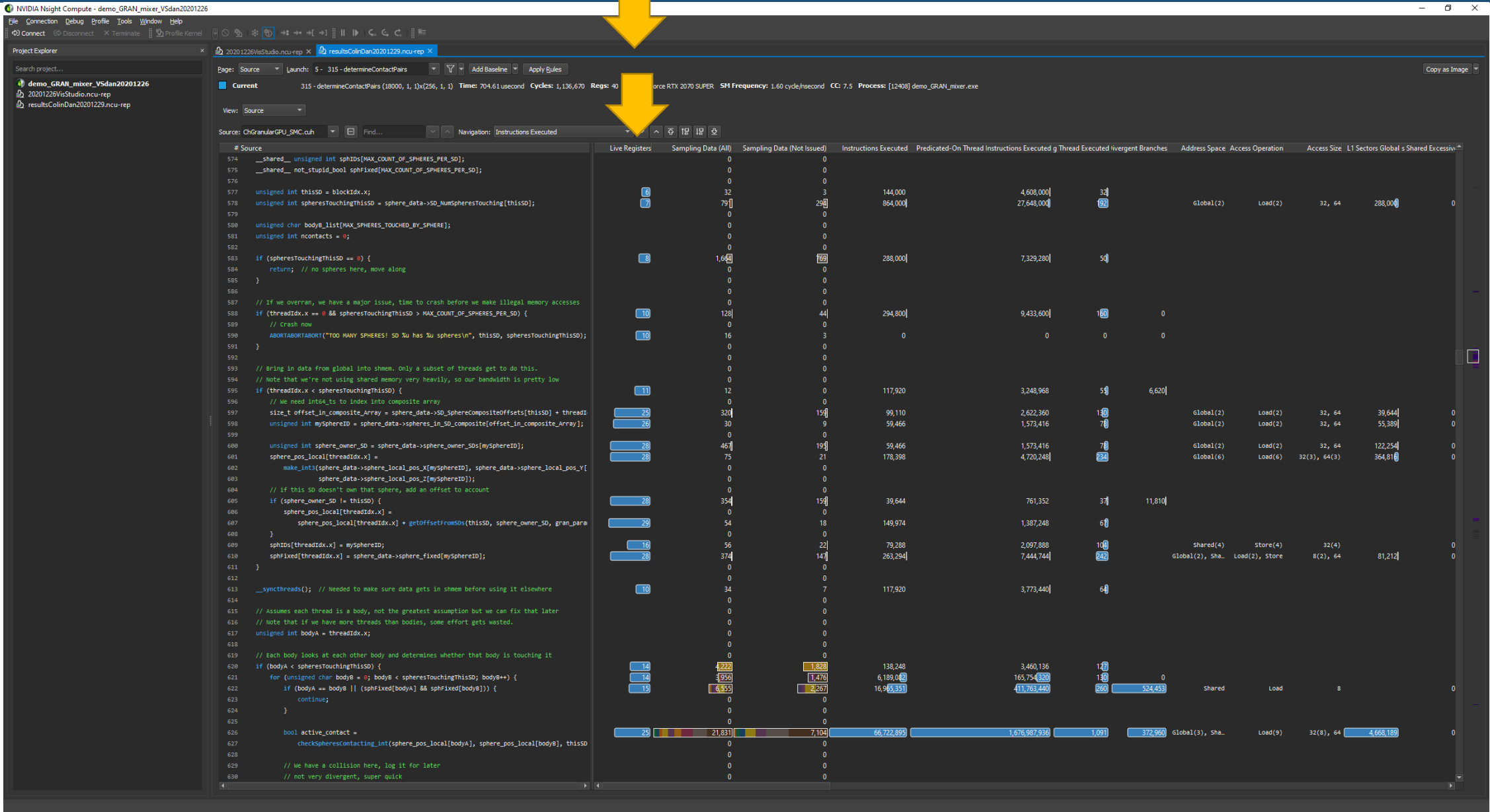


Possible per-block partitioning scenarios of the RF available on the SM

Live registers



[snapshot from NVIDIA's Nsight]



From the left field...

- Q: Who do you think controls/decides the register allocation when executing a kernel?
- A: The compiler